EE 419 Project 2 Design Document

Geeoon Chung and Nate Snyder

Our project generally followed the design laid out by the specifications with some creative changes. Every one of the messages sent between devices had a one byte header, starting with either “A”, “L”, or “D” for keep alive, link-state advertisement, and kill. The contents of the message were serialized in JSON.

Keep Alive Message Implementation

For our implementation of the keep alive message, we kept it simple. A separate thread would continuously be running, sending a keep alive message every ALIVE_SGN_INTERVAL (0.5) seconds. To prevent threads from being blocked (e.g., peer is down, message dropped, etc.), we created a wrapper for the sockets, *_send_msg*. When *_send_msg* is called, a new thread is created which opens a socket for the target and sends a message to the target. So for keep alive, we create a separate thread for each peer we want to send a keep alive to, allowing the thread to send the keep alive messages without blocking for too long. The node sending the keep alive message will include its name so that the receiver can get the name of the node. It also includes the time that the message was sent (from the sender), so the timeouts can be more accurate.

Link-State Advertisement (LSA) Implementation

For our implementation of the link-state advertisement, we also kept it simple, and mostly adhered to the specifications. Each link-state advertisement contains the name, uuid, port, peers, and sequence number of the node sending the advertisement. The reason for some of the less link-state related values (like port) is to allow for adding new peers. When these link-state advertisements are received, the receiver will first check if the link-state originated from itself or if the sequence number is old (already received an LSA of the same sequence number or newer). If either of these conditions are met, the LSA packet is dropped. Otherwise, it is forwarded to all peers of the receiver (re-flood), then the global state of the network topology is updated to fit the new LSA, and if if the LSA is from a peer, then it is added as a neighbor and a new LSA is generated for the receiver (due to an update to its peers).

Libraries Used

Only the Python standard library was used: socket, uuid, json, heapq, and math.

Extra Capabilities

We added the kill ability to our program. This allows users to run the “kill” command to automatically take down a node. On death, it floods its peers with a kill message and its UUID. If a receiver gets a kill message, and the originating node is in its peers or network topology data structure, then it will forward the message to all its peers (re-flood the message). The receivers will remove the originator of the death message from both its peers and/or network topology.

Additionally, for testing, we created a simple network of Docker containers running different nodes. Tmux was used to interact with each of the test nodes separately and run commands on them.